Intuition Engine Programmer's Reference Guide

Preface

Intuition Engine is a modern 64-bit RISC machine: a re-imagining of Commodore/Atari/Sinclair/BBC/Amstrad/IBM 8/16/32-bit home-computer ideas from the 1980s and 1990s. It is built as an homage to that era of home computing, while remaining one Intuition Engine computer with a shared memory bus. Processors, video chips, sound engines, DMA hardware, file devices, input devices, and control registers all sit on the same backplane. When you move from VideoChip to VGA, or from SID to POKEY, or from IE64 to 6502, you are not changing computers. You are programming another card on the same bus.

This guide begins at the BASIC prompt because that is the quickest way to touch the machine. You will type short programmes, inspect memory with `PEEK`, change hardware with `POKE`, and then use IE Mon to enter machine-code bytes directly. The examples are written for that path. They do not require an assembler, a build command, or a second machine to understand what is happening.

The book is also a reference. Chapter 2 is deliberately a vocabulary chapter, so skim it on a first reading and return to it when a keyword needs checking. The real climb continues in Chapter 3, where the screen becomes visible, then through sound, memory, processors, I/O, and whole-machine workflows. Part VII is different: it is a guided demo-programming course that uses the same registers and commands to build frame loops, rotozoomers, music-synchronised sections, copper presentation, and complete intro structure. Part VIII is an advanced case study in a large game port. It shows how the same machine model scales when one programme uses an M68K game core, an IE coprocessor, Voodoo rendering, native audio, file storage, input, and profiling as one system.

Keep one rule in mind as you read: every chip and every CPU is part of the same Intuition Engine.

Contents

Part III - Programming Sound and Music

Part IV - BASIC to Machine Language

Part V - Input / Output Guide

Part VI - Whole-Machine Project

Part VII - Demo Programming